3D Point World Models: Point Completion Enables More Accurate Dynamics Learning

***Abstract*—Learning predictive models of the world enables robotic control through planning, potentially allowing robots to improvise solutions on new tasks. However, large video-based dynamics models lack explicit 3D spatial structure and suffer from geometrically inconsistent long-term rollouts with compounding errors. Emerging 3D dynamics models based on partial point clouds improve geometric consistency but remain sensitive to occlusions and accumulated prediction drift. To address these challenges, we present 3D Point World Models (3DPWM) – a task-agnostic world model that operates entirely in 3D space by first completing partial point clouds and then learning action-conditioned dynamics in this completed 3D scene. By operating on completed geometry, 3DPWM enables reliable long-horizon rollouts and more accurate cost evaluation for model-based planning while supporting adaptation to new tasks. Experiments across different robotic embodiments and tabletop manipulation benchmarks demonstrate that 3DPWM achieves substantially higher rollout accuracy compared with state-of-the-art 3D world models, including PointWorld, supports both open-loop and closed-loop planning, and enables successful sim-to-real transfer. Webpage: https://3dpwm-anonymous.github.io/**

***Index Terms*—3D dynamics learning, point completion**

I. Introduction

Embodied agents acting in physical environments benefit from having a 3D understanding of the world. To endow such agents with capabilities to reason about their surroundings and plan their actions accordingly, they should be able to not just interpret the current state of the world, but also predict the outcomes of an action that they would take. In other words, agents should learn a reliably accurate world model [1] that facilitates planning in the real world.

Prior work in world model-based robot control has predominantly learned *task-specific* world models from 2D images [2]–[6], using them for reinforcement learning or model based planning [7]–[9]. These models typically train both transition and reward functions, which must be fine-tuned or re-trained when adapting to new tasks. More recently, large video models [10]–[12] have emerged by training on vast internet-scale video datasets to learn transition dynamics. However, these models face two critical challenges for real-world deployment. First, they are prone to hallucinating unrealistic or implausible future frames. Second, to enable action execution in physical environments, they require a learned inverse kinematics (IK) model to infer actions from generated videos [13]–[15]. Learning such robust and generalizable IK models remains challenging due to the inherent ambiguity in action-state correspondences – for example, the same visual motion of a robot arm lifting an object could correspond to very different torque commands depending on the object's (unobserved) mass or friction properties.

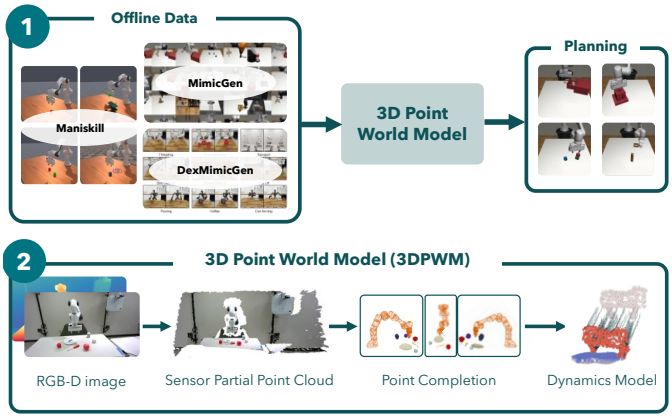


Fig. 1: *Overview.* We propose **3DPWM**, a task-agnostic world model trained from demonstrations and deployed for planning via model-predictive control. Given a single-view RGB-D observation, the system constructs a partial point cloud, performs point completion, and then rolls out action-conditioned trajectories in 3D space.

Another class of models that has been explored is 2.5D video models [16]–[18] that jointly predict geometric properties such as depth and surface normals alongside RGB frames. They are more grounded in the physical 3D world, however, since such models process these additional features in a 2D fashion, they rely on implicit geometric reasoning and still remain prone to multi-view inconsistencies. For example, normals and depth inferred from one camera viewpoint may not match those inferred from another. Furthermore, owing to the dynamics model operating on partial RGB(-D) observations, both classes of models suffer from compounding rollout errors, which hamper reliable long-horizon prediction and planning.

Recent work learns explicit 3D world models from partial point clouds [19], [20], particles [21]–[23], or Gaussian splats [24], enabling planning by simulating how scene geometry evolves under actions. However, these approaches suffer compounding errors in occluded scenes due to partial-view inputs [19], [20], [24]. Meanwhile models trained on complete point clouds in simulation [22], [23], [25] struggle in real-world deployment where complete geometry is unavailable. We bridge this gap by designing a framework that integrates point cloud completion into 3D world modeling, enabling a world model to operate on complete 3D geometry in both simulation and the real world while retaining the benefits of training on complete geometry.

We propose 3D Point World Models (**3DPWM**) – a system

to train 3D point cloud world models in a *task-agnostic* fashion, allowing training on diverse (non-expert) trajectories. Different from prior work that learns from a vector latent space [20] or only from partial point clouds [19], we emphasize the importance of complete 3D geometry and demonstrate that it is much easier for the dynamics model to reason about the center of gravity and impact in occluded areas. To that end, we develop a point completion pipeline tailored to robot manipulation settings and introduce an action-conditioned 3D world model that operates on complete 3D geometry, achieving state-of-the-art rollout accuracy across multiple robot embodiments Our key insight in this work is that complete point clouds offer better training regime for point clouds based world models, which in turn (a) facilitates reliable computation of cost functions during deployment for planning, and (b) adaptation to novel unseen tasks that were not present in the dynamics training distribution. Through results across various tasks, we show that the progress in scene completion [26] and, in particular, 3D point cloud completion [27], [28], has reached sufficient maturity to be integrated into robotic systems. A high level overview of the system is shown in Figure 1.

Contributions. We summarize our main contributions:
– We introduce **3DPWM**, a framework that integrates point completion with 3D world modeling to enable dynamics prediction over complete geometry, demonstrating state-of-the-art rollout performance across robot manipulation scenes.
– We provide a training recipe combining data augmentation, multi-step rollout losses and register tokens, that greatly improve the performance of **3DPWM**.
– We show that **3DPWM**'s accurate long-horizon rollouts across different robot embodiments enable better closed-loop and open-loop planning than baselines, as well as adapting to a novel combination of tasks.
– Finally, we show effective sim-to-real transfer of the system on tabletop manipulation tasks where **3DPWM** performs $2.5\times$ better than baselines.

II. Related Works

Point Cloud World Models. Several prior works have explored learning dynamics models directly from meshes [22], [29] and point clouds using graph neural network (GNN), for both deformable and rigid-body systems [23], [25], [30]–[33]. More recently, to improve the accuracy of point cloud-based dynamics modeling, transformer architectures [34] have been adopted [19], [35]. In particular, ParticleFormer [19] introduced a transformer-based, object-centric point cloud dynamics model. However, these approaches differ from the proposed system in two key aspects: (a) the control capabilities demonstrated in prior methods are largely limited to visual planning on push-like tasks. In contrast, our approach supports more complex manipulation behaviors, including pick-and-place. (b) Prior work that operates on complete point clouds derived from meshes has primarily been evaluated in simulated environments [22], [23], [25], [29]. When training such dynamics models in real-world settings with partial point clouds [19], [30], [31], Chamfer distance is often used to handle points that disappear or appear due to occlusion and disocclusion. This results in weaker supervision signals for world model learning. In contrast, we train our world model using complete point clouds in simulation only and demonstrate sim-to-real transfer, when paired with a point cloud completion module, enabling effective 3D world modeling with complete geometry in real-world environments.

In PointWorld [36], the authors train a single, large 3D point cloud dynamics model on real-world DROID dataset [37] and show the effectiveness of their world model on several real-world tasks. Our model has a couple of notable differences with it: a) PointWorld operates on partial point clouds, resulting in rollouts with missing geometry when observations are heavily occluded, whereas our model reconstructs complete object geometry as long as objects are partially observable. We demonstrate that maintaining complete geometry during rollouts leads to better training for world models and enables more accurate cost functions for planning. b) PointWorld demonstrates relatively shorter-horizon rollouts (30 steps for planning, 10 steps for rollout visualization), while our work demonstrates substantially longer-horizon rollouts (up to 300 steps) that remain stable and accurate. c) The manipulation scenarios in PointWorld primarily involve collision-free settings (except for the microwave door open task) in which an object is already grasped by the robot. In contrast, our work focuses on prehensile manipulation and supports planning from initial states before the robot has made contact with any object, in both open-loop and closed-loop control settings.

Latent World Models. While image based latent world models such as Dreamer [2]–[4], [9] and TDMPC [6], [38] learn task specific dynamics models, DINO-WM [39], V-JEPA [40] learns dynamics over frozen DINOv2 patch features and plans by optimizing action sequences toward goal features, sharing our offline, task-agnostic, reward-free setting. However, because its rollouts live in a 2D latent space, cost functions cannot be written over scene geometry, which is what enables **3DPWM**'s completion-based costs.

Point Cloud Completion. Early work on point cloud completion focused on reconstructing a single object from a partial point cloud in a canonical object space [27], [41]–[43], which largely limited these methods to synthetic settings and made real-world deployment challenging, since real-world point clouds often contain multiple objects and are not aligned with a canonical object space. More recent work has shifted toward instance-level point cloud completion from partial point clouds of multi-object scenes, rather than single-object reconstruction [28], [44]–[46]. To the best of our knowledge, this is the first work to combine point cloud completion with 3D world modeling for real-world manipulation, enabling more accurate world models based on complete geometry.

III. Methodology

Problem Formulation. Our system takes in robot proprioception (with joint angles $\mathbf{q}$) and a single-view partial point cloud (obtained from an RGB-D camera) as the observation $\mathcal{O}$

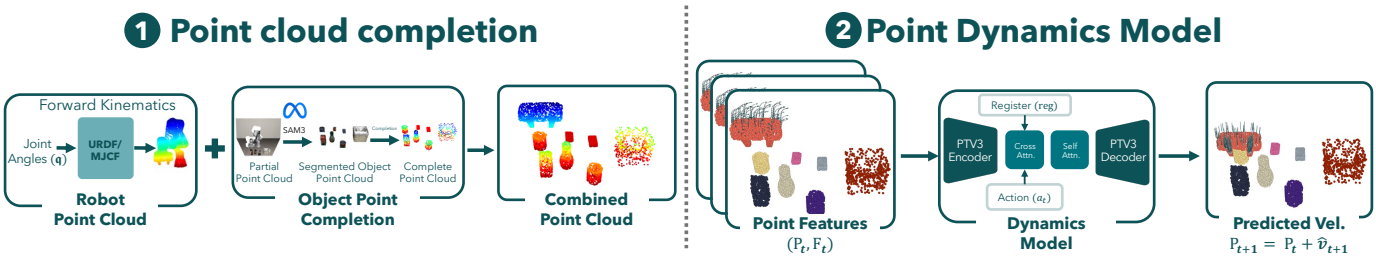


Fig. 2: *System Overview.* **3DPWM** consists of ❶ *Point completion*, which takes a single-view calibrated RGB-D input and robot joint angles to (a) sample end-effector points via forward kinematics on the robot URDF, (b) segment scene objects with SAM3 and complete their geometries using a point completion model [28], and (c) merge robot and object points into a full scene point cloud; and ❷ *Point dynamics*, which trains an action-conditioned model to predict next-step point velocities $\hat{v}$.

at every step. Given this partial observation, we first complete the point cloud to obtain the complete observation of the visible scene $\mathcal{O}^{\text{complete}}$. Then, given an action $a \in \mathcal{A}$, we train the transition function $\mathcal{T} : \mathcal{O}^{\text{complete}} \times \mathcal{A} \rightarrow \mathcal{O}^{\text{complete}}$ that predicts the next observation. The goal is to use this learned dynamics model with a planner to roll out trajectories and optimize a desired cost objective.

Obtaining Complete Point Clouds. While complete point clouds are available from object meshes during training in simulation, at deployment time we obtain complete point clouds as follows: We first segment each visible object in the scene using SAM3 [47] with point prompts generated by VLM [48], and back-project the depth map to recover partial object point clouds. We then apply point cloud completion to produce completed object point clouds, $\mathbf{P}_t^{\text{obj}}$. We fine-tune the model from [28], which is pre-trained on the ShapeNet dataset, using objects from the YCB [49] dataset. This model improves upon previous point completion models in that it does not require knowledge of the complete object center, a shortcoming that prevents previous completion models from being used here.

We revisit the data preprocessing and augmentation strategy. In [28], point cloud completion is formulated for upright objects in the scene by aligning each partial object point cloud input (extracted via instance segmentation) to a canonical upright orientation along the y-axis. While this simplifies learning, it restricts the ability of the model to generalize to dynamic scenarios in which objects may appear in arbitrary orientations, leading to degraded completion performance. To address this limitation, we apply rotation augmentation using all three axes (x, y, and z) during both pre-training on ShapeNet and fine-tuning on YCB [49]. Additionally, we apply random jittering to simulate depth noise from depth sensors. To further mitigate sensor depth noise, particularly near object boundaries and on glossy surfaces, we follow [50] to clean up boundary-related depth noise and apply a median filter to suppress noisy depth values within each object.

To obtain complete robot point clouds, given $\mathbf{q}_t$ and the robot URDF, we use forward kinematics to compute the robot's mesh configuration and sample points from it to obtain the point cloud of the robot $\mathbf{P}_t^{\text{robot}} = \text{FK}(\mathbf{q}_t, \text{URDF})$. Since the action space operates in the end-effector frame, we only sample points from the end-effector mesh as shown in Fig. 2. The final observation is a combination of the robot end-effector points and complete object point clouds; $\mathbf{P}_t = \mathbf{P}_t^{\text{robot}} \cup \bigcup_{\text{obj}} \mathbf{P}_t^{\text{obj}}$.

3D Point World Model. Given the complete point cloud of the scene $\mathbf{P}_t$, we featurize each point with its current spatial location as well as both current and previous timestep velocities – $\mathbf{F}_t = [\mathbf{x_t}, \mathbf{y_t}, \mathbf{z_t}, \mathbf{v_t}, \mathbf{v_{t-1}}]$. Since we train on datasets collected in simulation, we can obtain point-to-point correspondences across time to compute the velocities. In the real-world, these correspondences can be obtained via point tracking [50]. To train the dynamics model, this observation representation: $(\mathbf{P}_t, \mathbf{F}_t)$ is encoded using a point cloud backbone. Specifically, we use Point Transformer V3 (PTV3) [51], a state-of-the-art point cloud encoding architecture to obtain a point-based latent representation $\mathbf{z}_t \in \mathbb{R}^{M \times d}$, where M is the number of points after downsampling and d is the dimensionality of each point feature. At this bottleneck level, we induce action conditioning into the network by first encoding the action a_t with a small MLP; $\mathbf{z}_t^{\text{act}} = \text{MLP}(a_t)$. We then perform cross-attention [34] where along with the action embedding, we also include a learnable *register* token [52]. These serve as keys and values, and the point features $\mathbf{z}_t$ serve as queries. This is to ensure that action-relevant points (i.e., corresponding to the robot) can attend to action commands, while action-irrelevant points (i.e., scene points not interacting with the robot) can attend to the register token ($\mathbf{reg}$), which effectively acts as a dummy key. We find this cross attention design with a register token to be important for performance. The cross attention layer is followed by self-attention layers to propagate the effect of the action across all scene points:

$$\tilde{\mathbf{z}}_t = \text{CrossAttn}\big([\mathbf{z}_t^{\text{act}}, \mathbf{reg}],\ \mathbf{z}_t\big), \qquad \mathbf{z}'_t = \text{SelfAttn}(\tilde{\mathbf{z}}_t)$$

Finally, the resulting point cloud representation is decoded using PTV3's point decoder to obtain a feature representation at the original point cloud resolution. A lightweight MLP is then applied to the per-point features to predict per-point velocities at the next timestep, $\hat{v}_{t+1}$. The next timestep point cloud can be obtained by:

$$\hat{\mathbf{P}}_{t+1} = \mathbf{P}_t + \hat{\mathbf{v}}_{t+1} \tag{1}$$

Long-horizon rollouts are produced by recursively applying the dynamics model and Eq.1 where the input observation for

timestep $t+i$ is:

$$(\hat{\mathbf{P}}_{t+i}, \hat{\mathbf{F}}_{t+i}) = (\hat{\mathbf{P}}_{t+i}, [\hat{\mathbf{x}}_{t+i}, \hat{\mathbf{y}}_{t+i}, \hat{\mathbf{z}}_{t+i}, \hat{\mathbf{v}}_{t+i}, \hat{\mathbf{v}}_{t+i-1}]) \quad (2)$$

Training Objective. Given access to point-to-point correspondences in our dataset, we train the **3DPWM** using a Huber loss [53] between the predicted and ground-truth per-point velocities. This provides direct supervision on the dynamics, in contrast to prior work [19] that relies on Chamfer or Hausdorff distances. We empirically validate the benefit of this formulation in Section IV. Rather than providing single step supervision, we train **3DPWM** based on multi-step rollouts – iteratively applying our model to produce the following H time steps induced by an action sequence $a_t, ..., a_{t+H}$:

$$\mathcal{L} = \frac{1}{H} \sum_{h=1}^{H} \text{Huber}\Big(\hat{\mathbf{v}}_{t+h}, \mathbf{v}_{t+h}\Big), \quad (3)$$

We find that the multi-step rollout loss functions significantly reduce compounding errors (Sec. V).

Planning with 3DPWM. We employ **3DPWM** in a model predictive control framework (MPC) using a sampling based MPPI planner [54]. We roll out $\mathcal{K}$ trajectories at each step and optimize with a pre-defined cost function. This cost function is defined in the observation space of the complete point cloud $\mathbf{P}_t$, similar to how a reward function for a downstream task is written with a simulator.

IV. Experiments

We show the efficacy of **3DPWM** along 4 fronts: (1) Can **3DPWM** lead to reliable and accurate long-horizon rollouts on diverse embodiments?; (2) How well can **3DPWM** be used to effectively plan for robotic manipulation tasks?; (3) Can **3DPWM** be used for adaptation to unseen tasks; and (4) Can **3DPWM** be transferred to real-world manipulation tasks?

Data Collection. For training the dynamics modeling, we collect data from a mix of expert demonstrations and augment the demos with random actions to cover a larger portion of the state space. Specifically, with a probability of 0.2, we induce randomness in the expert trajectory by uniformly selecting a timestep i and executing random action from step i. All actions are recorded in delta end-effector space. = Instead of completely using random trajectories from the beginning, our rationale to have this kind of augmentation is to induce novel collision dynamics that are unseen in the expert demonstration – such as trying to execute a task and failing mid-way. Early on in the project, we observed that this kind of augmentation led to a dynamics model that was more performant on planning. Specifically on `PickYCB`, this data augmentation boosted the performance of closed-loop planning from 58% to 65%. On `PickCube` the difference was similar ($\sim$ 83% $\rightarrow$ 90%). Additional details are in Sec. VIII-A.

A. *Evaluating accuracy of long-horizon rollouts.*

Setting. To evaluate **3DPWM**'s rollout capability, we train on seven datasets spanning two end-effector embodiments (Franka parallel gripper and GT-1 dexterous hand). For Franka, we use `PickCube`, `StackCube`, and `PickYCB` from ManiSkill [55], and `MugCleanup` from MimicGen [56]. The PickYCB task requires the robot to be able to pick up any of the 78 objects from YCB dataset, and is considered a difficult task for planning. For the dexterous suite, we use `BoxCleanup`, and `DrawerCleanup` from DexMimicGen [57]. Additional details are described in Appdx. VIII-I.

Baselines. We compare **3DPWM** against three baselines:
– *PointWorld*: PointWorld [36] is the closest work to ours, but differs in two key aspects: 1) it operates on partial point clouds and 2) it uses action flows rather than end-effector (EE) actions as its action space. For a fair comparison, we convert our EE actions into action flows using inverse kinematics.
– *ParticleFormer*$^+$: A modified version of ParticleFormer [19] that adopts the same PTV3-based encoder–decoder as ours for a fair comparison. Note that this is a significantly larger network than the one in [19] which only had 3 self-attention layers. The model is trained using the hybrid loss proposed by the authors and operates on object-centric partial point clouds.
– *Partial* **3DPWM**: A variant of **3DPWM** in which the point completion module is removed; equivalently, this corresponds to ParticleFormer$^+$without object-centric features and trained with Chamfer and Hausdorff losses as done in [19].
For all methods, the model observes the first three point clouds of each test trajectory and then rolls out to predict future point clouds, conditioned on the actions from the test demonstration.

Metrics. We report the $\mathcal{L}_1$ Chamfer distance between rolled-out and ground-truth point clouds in Tab. I. However, this global metric does not directly reflect planning utility, as drift in critical points (e.g., the tool center point or an object's center of mass) can be masked by averaging. We therefore additionally evaluate downstream task rewards on rolled-out point clouds, comparing them to ground-truth simulator rewards using Pearson correlation (r). For ManiSkill tasks, we use the benchmark's shaped rewards; for MimicGen and DexMimicGen, we extract reward functions using VLM (details in Appdx. VIII-C). For all partial baselines, we use the visible-point mean as the CoM, providing a fair estimate under the available partial observation without assuming access to the privileged unobserved geometry.

Results. Across all tasks, **3DPWM** achieves significantly lower Chamfer distance (average rollout lengths reported in Tab. I). ParticleFormer$^+$, Partial **3DPWM**, and PointWorld are competitive with each other across different tasks, but all exhibit substantially higher rollout errors than **3DPWM**. PointWorld is competitive on the shorter parallel gripper tasks, but its performance degrades on long-horizon tasks and with the dexterous hand end-effector. Similarly, its reward correlation drops sharply on the dexterous hand tasks. Overall, these results show that learning dynamics from partial point clouds is challenging as increasing occlusions limit the model's ability to infer complete scene geometry; explicit completion removes the need for the dynamics model to implicitly reason about occluded points. Consistently, **3DPWM** outperforms all baselines on reward corr., further corroborating the benefit of point completion. Qualitative results are shown in Fig.



Fig. 3: *Qualitative rollout results of* **3DPWM**. Action-conditioned rollouts of **3DPWM** over time for five tasks. Ground-truth trajectories are shown in **purple**, while predicted rollouts are shown in **teal**. The second column (with purple border) shows the initial observed point cloud, and all subsequent point clouds are generated by rolling out the model using actions from the ground-truth trajectory. A higher overlap between the purple and teal points indicate better accuracy of world model rollouts.

3. Results with additional baselines, including a GNN-based baseline, are provided in the Appendix.

B. *Model-based Planning.*

Setting. We evaluate **3DPWM** under model-based planning using MPPI [54] in both *open-loop* and *closed-loop* settings on `PickCube`, `StackCube`, and `PickYCB`. A key motivation for learning a dynamics model is to use it as a proxy simulator in the real world, where ground-truth observations are unavailable, making accurate open-loop control important. In the open-loop setting, the system observes only the initial point cloud and plans actions for the entire horizon, serving as a stress test to assess whether **3DPWM** can function as a robust simulator for planning. To obtain the cost function for planning, following works like Eureka [58], and Dr. Eureka [59] - we prompt Gemini 2.5 Flash VLM [60] to generate a cost function given the annotated objects in the scene.

Baselines. Beyond ParticleFormer^{+}and Partial **3DPWM**, we evaluate **3DPWM** with ground-truth and fine-tuned point completion to quantify the impact of learned completion on planning success. This comparison quantifies the impact of learned point completion on planning success. In addition, we introduce an analytic transition model (denoted by `FK`) that uses forward kinematics to predict future EE poses and compute their costs. This comparison evaluates the importance

TABLE I: Chamfer distance and reward correlation across embodiments and tasks (Best and second best methods are highlighted).

End-effector	Task	Avg. len.	Chamfer L1 (↓) PF⁺	Chamfer L1 (↓) Partial 3DPWM	Chamfer L1 (↓) Point World	Chamfer L1 (↓) 3DPWM	Reward Corr (r) (↑) PF⁺	Reward Corr (r) (↑) Partial 3DPWM	Reward Corr (r) (↑) Point World	Reward Corr (r) (↑) 3DPWM
Parallel Gripper	PickCube	66	0.080	0.085	0.071	0.009	0.61	0.57	0.69	0.86
	StackCube	99	0.090	0.097	0.094	0.014	0.55	0.55	0.53	0.83
	Pick YCB	143	0.089	0.089	0.064	0.028	0.53	0.51	0.68	0.85
	Mug Cleanup	336	0.130	0.132	0.174	0.022	0.40	0.45	0.35	0.79
Dexterous Hand	Box Cleanup	218	0.132	0.149	0.211	0.022	0.30	0.32	0.11	0.83
	Drawer Cleanup	274	0.273	0.246	0.351	0.058	0.29	0.25	0.04	0.77

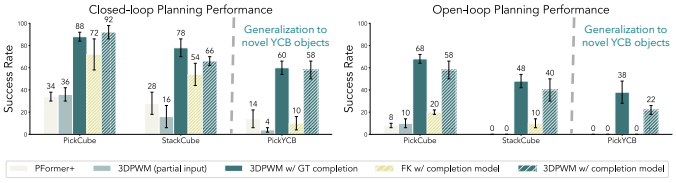


Fig. 4: *Planning success rate*: **(Left)** Closed-loop planning success. **3DPWM** achieves the highest success across all tasks. **(Right)** Open-loop planning success on the same tasks, where **3DPWM** significantly outperforms prior methods due to higher-fidelity rollouts.

of learning a dynamics model for the task.

Metrics. We report success rates over 100 MPPI runs, rolling out 200 samples with horizon $H = 10$ at each step, for a maximum of 100 time steps.

Results. Fig. 4 reports performance on three tasks under closed-loop and open-loop planning.

Closed-loop planning. With GT completion, **3DPWM** achieves 88% success on `PickCube`, decreasing to 78% on `StackCube` and 60% on `PickYCB` as task complexity increases. Learned point completion introduces a 2-10% drop on the harder tasks due to noisy completions (see Appendix VIII-D). Both ParticleFormer$^+$and Partial **3DPWM** exhibit low success rates on partial point clouds, with ParticleFormer$^+$performing slightly better, likely due to its explicit object-centric inductive bias. Interestingly, the analytical forward kinematics model combined with point completion achieves good success rates on both `PickCube` and `StackCube`. However, its performance decreases on the more diverse and challenging `PickYCB` task. This suggests that while simple kinematic models can be sufficient for relatively simple objects and interactions, handling a diverse set of objects benefits from reasoning about the consequences of multiple MPPI-generated actions over longer horizons, where **3DPWM** provides a clear advantage.

Open-loop planning. All methods show a substantial drop relative to closed-loop, but **3DPWM** with ground-truth completion—and to a lesser extent with learned completion—still achieves non-trivial success rates. In contrast, ParticleFormer$^+$and Partial **3DPWM** largely fail (1/20 on `PickCube`, 0% elsewhere), due to low-quality rollout predictions leading to inaccurate cost estimation.

C. *Adaptation to unseen long-horizon tasks.*

Setting. We train **3DPWM** on MimicGen [56] demonstrations for the `Coffee` and `Mug Cleanup` tasks and evaluate

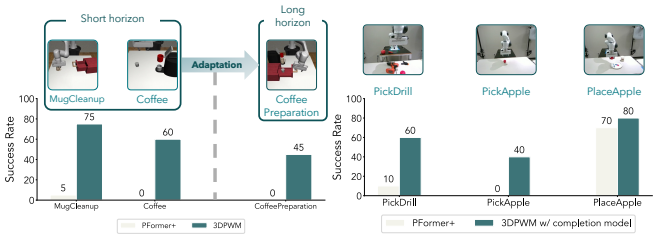


Fig. 5: **(Left)** *Adaptation to unseen tasks*: We train **3DPWM** dynamics on demonstrations generated on `MugCleanup` and `Coffee` tasks and show that it can be used to adapt to a much long-horizon task of `CoffeePreparation` and achieving 45% success rate. **(Right)** *Sim2Real*: We evaluate sim-to-real transfer of **3DPWM** on `PickApple`, `PickDrill`, and `PlaceApple`. The static third-person camera setup and objects are shown in the top-right. While **3DPWM** and ParticleFormer$^+$perform similarly on the easier placing task, **3DPWM** significantly outperforms on the picking tasks.

on `Coffee Preparation`, a long-horizon task requiring the robot to place a mug on the coffee machine, retrieve a pod from a drawer, load it, and close the lid (Fig 5). This tests whether **3DPWM** can compose individually learned dynamics into extended task sequences. We use ParticleFormer$^+$ as a baseline to assess whether point completion aids generalization to new tasks.

Results. On short-horizon tasks, **3DPWM** achieves 75% on `MugCleanup` and 60% on `Coffee`, with the latter's lower performance likely due to precise placement requirements. ParticleFormer$^+$ performs poorly on both due to incorrect dynamic predictions. On the long-horizon `CoffeePreparation`, chaining the subtasks leads to a 15% drop for **3DPWM** from compounding execution errors, while ParticleFormer$^+$ fails entirely.

D. *Sim2Real transfer of* 3DPWM.

Tasks. We evaluate sim-to-real transfer on tabletop *Pick* and *Place* tasks using a Franka Panda with a parallel-jaw gripper: `PickApple`, `PickDrill`, and `PlaceApple` (placing a toy apple onto a plate), with randomly placed distractors. Task setups and objects are shown in Fig. 5.

System Design. We use a single-view RGB-D observation from an Intel RealSense D455 camera, extrinsically calibrated to align with the Franka base and ManiSkill2 coordinate system [61]. Target objects are segmented using SAM3 [47] with text prompts and back-projected into 3D partial point clouds, which are then completed using a fine-tuned completion module [28]. For dynamics, we deploy the best `PickYCB` checkpoint from simulation *without* real-world fine-tuning. Planning uses MPPI with 30–50 step open-loop rollouts executed in chunks. Additional details are in Appd. VIII-B; robustness to camera viewpoints and lighting is reported in Appdx. VIII-E.

Baselines & Metrics. Since Partial **3DPWM** performs comparably or worse than ParticleFormer$^+$in simulation, we compare only against ParticleFormer$^+$in real-world experiments, report-

TABLE II: Ablation studies of **3DPWM** using Chamfer distance between GT and rollouts. We evaluate the effect of the # register tokens and prediction horizon H. The best-performing configuration is , while the closest competitive configuration without a significant performance loss is underlined.

Task	# Register tokens			Horizon H			
	0	1	2	3	5	10	15
PickCube	0.013	0.009	0.009	0.068	0.038	0.021	0.009
PickYCB	0.052	0.028	0.027	0.161	0.117	0.044	0.028
Mug Cleanup	0.178	0.022	0.020	0.102	0.036	0.028	0.022

ing success rates over 10 trials per `method, task` pair.

Results. Results are shown in Figure 5. On `PlaceApple`, both methods achieve comparable success (70–80%), as the plate's large support surface yields low placement failure rates. On `PickApple` and `PickDrill`, **3DPWM** substantially outperforms ParticleFormer^{+}, which we attribute to improved object geometry from point completion. Remaining failures primarily arise from biased center-of-mass estimation (see Appdx. VIII-D).

V. Ablation Study & Discussion

We examine two key design choices in **3DPWM**: the role of *register tokens* and the multi-step rollout length for training.

Register Tokens. We ablate the number of register tokens [52] (#reg $\in \{0, 1, 2\}$) used as keys in cross-attention alongside the action encoding $\mathbf{z}_t^{\text{act}}$ (Table II). Increasing from #reg $= 0$ to #reg $= 1$ yields substantial gains, while additional tokens show diminishing returns. We use #reg $= 1$ for all experiments. As discussed in Sec. III, we hypothesize that register tokens help the model focus on task-relevant points in $\mathbf{z}_t$ during cross-attention.

Multi-step supervision. We ablate the supervision horizon $H \in \{3, 5, 10, 15\}$ (Table II). Longer horizons consistently improve dynamics modeling at the cost of increased training time due to autoregressive rollouts. We use $H = 15$ for all experiments.

Computational time. On a standalone RTX-4090, planning one action takes $\sim$1.07s, with tasks requiring 4-5 open-loop chunks of 20 actions. The main latency source is sequential dynamics rollout in observation space, which could be sped up via CUDA kernels supporting chunked multi-step prediction. Training takes 12–24 hours on a single L40 GPU, depending on dataset size, with larger-point datasets (`MugCleanup`) and higher-dimensional action spaces (`TwoArmBoxCleanup`, `TwoArmDrawerCleanup`) requiring closer to 24 hours.

VI. Limitations

3DPWM combines several modular components—VLMs, foundation segmentation models (SAM3), point completion, dynamics modeling, and model-based planning. This modular design enables flexible integration of these components, but also leaves room to improve robustness and efficiency. In our Sim2Real experiments, segmentation occasionally struggled with partially occluded or gripper-held objects, and point completion would likely benefit from training on more diverse, scene-level partial point clouds. Additionally, the sequential dynamics rollouts used for planning introduce latency and currently favor quasi-static tasks. Addressing these limitations—for example, through scene-centric completion training, chunked multi-step prediction [36], and lower-dimensional action representations—could further improve the robustness and scalability of **3DPWM**.

VII. Conclusion

In this work, we demonstrate that world models based on explicit point cloud completion lead to improved rollout quality. **3DPWM** achieves better predicted geometry, reward correlation, and downstream planning performance across simulated and real tasks, including adaptation to new long-horizon tasks. Our analysis suggests tighter integration of perception components and reduced planning latency are critical directions for further improvement.

References

[1] D. Ha and J. Schmidhuber, "World models," *arXiv preprint arXiv:1803.10122*, 2018.

[2] D. Hafner, T. Lillicrap, J. Ba, and M. Norouzi, "Dream to control: Learning behaviors by latent imagination," *arXiv preprint arXiv:1912.01603*, 2019.

[3] D. Hafner, T. Lillicrap, M. Norouzi, and J. Ba, "Mastering atari with discrete world models," in *ICLR*, 2021.

[4] D. Hafner, J. Pasukonis, J. Ba, and T. Lillicrap, "Mastering diverse domains through world models," *arXiv preprint arXiv:2301.04104*, 2023.

[5] P. Wu, A. Escontrela, D. Hafner, P. Abbeel, and K. Goldberg, "Daydreamer: World models for physical robot learning," in *CORL*, 2023.

[6] N. Hansen, H. Su, and X. Wang, "Td-mpc2: Scalable, robust world models for continuous control," in *ICLR*, 2024.

[7] K. Chua, R. Calandra, R. McAllister, and S. Levine, "Deep reinforcement learning in a handful of trials using probabilistic dynamics models," in *NeuRIPS*, 2018.

[8] A. Nagabandi, K. Konolige, S. Levine, and V. Kumar, "Deep dynamics models for learning dexterous manipulation," in *CORL*, 2020.

[9] D. Hafner, T. Lillicrap, I. Fischer, R. Villegas, D. Ha, H. Lee, and J. Davidson, "Learning latent dynamics for planning from pixels," in *International conference on machine learning*, 2019.

[10] N. Agarwal, A. Ali, M. Bala, Y. Balaji, E. Barker, T. Cai, P. Chattopadhyay, Y. Chen, Y. Cui, Y. Ding *et al.*, "Cosmos world foundation model platform for physical ai," *arXiv preprint arXiv:2501.03575*, 2025.

[11] J. Wu, S. Yin, N. Feng, X. He, D. Li, J. Hao, and M. Long, "ivideogpt: Interactive videogpts are scalable world models," in *NeuRIPS*, 2024.

[12] J. Ho, T. Salimans, A. Gritsenko, W. Chan, M. Norouzi, and D. J. Fleet, "Video diffusion models," in *NeuRIPS*, 2022.

[13] J. Liang, P. Tokmakov, R. Liu, S. Sudhakar, P. Shah, R. Ambrus, and C. Vondrick, "Video generators are robot policies," *arXiv preprint arXiv:2508.00795*, 2025.

[14] Y. Du, S. Yang, B. Dai, H. Dai, O. Nachum, J. Tenenbaum, D. Schuurmans, and P. Abbeel, "Learning universal policies via text-guided video generation," in *NeuRIPS*, vol. 36, 2023, pp. 9156–9172.

[15] S. Zhou, Y. Du, J. Chen, Y. Li, D.-Y. Yeung, and C. Gan, "Robodreamer: Learning compositional world models for robot imagination," *arXiv preprint arXiv:2404.12377*, 2024.

[16] H. Zhen, Q. Sun, H. Zhang, J. Li, S. Zhou, Y. Du, and C. Gan, "Tesseract: Learning 4d embodied world models," *arXiv preprint arXiv:2504.20995*, 2025.

[17] Y. Shang, X. Zhang, Y. Tang, L. Jin, C. Gao, W. Wu, and Y. Li, "Roboscape: Physics-informed embodied world model," *arXiv preprint arXiv:2506.23135*, 2025.

[18] X. Ren, T. Shen, J. Huang, H. Ling, Y. Lu, M. Nimier-David, T. Müller, A. Keller, S. Fidler, and J. Gao, "Gen3c: 3d-informed world-consistent video generation with precise camera control," in *CVPR*, 2025.

[19] S. Huang, Q. Chen, X. Zhang, J. Sun, and M. Schwager, "Particleformer: A 3d point cloud world model for multi-object, multi-material robotic manipulation," *Conference on Robot Learning*, 2025.

[20] S. Peri, I. Lee, C. Kim, L. Fuxin, T. Hermans, and S. Lee, "Point cloud models improve visual robustness in robotic learners," in *ICRA*, 2024.

[21] B. Ai, S. Tian, H. Shi, Y. Wang, T. Pfaff, C. Tan, H. I. Christensen, H. Su, J. Wu, and Y. Li, "A review of learning-based dynamics models for robotic manipulation," *Science Robotics*, 2025.

[22] K. R. Allen, T. L. Guevara, Y. Rubanova, K. Stachenfeld, A. Sanchez-Gonzalez, P. Battaglia, and T. Pfaff, "Graph network simulators can learn discontinuous, rigid contact dynamics," in *CORL*, 2023, pp. 1157–1167.

[23] C. Kim and L. Fuxin, "Object dynamics modeling with hierarchical point cloud-based representations," in *CVPR*, 2024.

[24] M. Zhang, K. Zhang, and Y. Li, "Dynamic 3d gaussian tracking for graph-based neural dynamics modeling," in *CORL*, 2024.

[25] Y. Li, J. Wu, R. Tedrake, J. B. Tenenbaum, and A. Torralba, "Learning particle dynamics for manipulating rigid bodies, deformable objects, and fluids," in *ICLR*, 2019.

[26] S. D. Team, X. Chen, F.-J. Chu, P. Gleize, K. J. Liang, A. Sax, H. Tang, W. Wang, M. Guo, T. Hardin, X. Li, A. Lin, J. Liu, Z. Ma, A. Sagar, B. Song, X. Wang, J. Yang, B. Zhang, P. Dollár, G. Gkioxari, M. Feiszli, and J. Malik, "Sam 3d: 3dfy anything in images," *arXiv preprint arXiv:2511.16624*, 2025.

[27] H. Zhou, Y. Cao, W. Chu, J. Zhu, T. Lu, Y. Tai, and C. Wang, "Seedformer: Patch seeds based point cloud completion with upsample transformer," in *European Conference on Computer Vision*, 2022.

[28] W. Khademi and L. Fuxin, "Point-based instance completion with scene constraints," in *ICLR*, 2025.

[29] M. Fortunato, T. Pfaff, P. Wirnsberger, A. Pritzel, and P. Battaglia, "Multiscale meshgraphnets," *arXiv preprint arXiv:2210.00612*, 2022.

[30] H. Shi, H. Xu, Z. Huang, Y. Li, and J. Wu, "Robocraft: Learning to see, simulate, and shape elasto-plastic objects with graph networks," *arXiv preprint arXiv:2205.02909*, 2022.

[31] H. Shi, H. Xu, S. Clarke, Y. Li, and J. Wu, "Robocook: Long-horizon elasto-plastic object manipulation with diverse tools," *arXiv preprint arXiv:2306.14447*, 2023.

[32] A. Longhini, M. Büsching, B. P. Duisterhof, J. Lundell, J. Ichnowski, M. Björkman, and D. Kragic, "Cloth-splatting: 3d cloth state estimation from rgb supervision," *arXiv preprint arXiv:2501.01715*, 2025.

[33] T. Tian, H. Li, B. Ai, X. Yuan, Z. Huang, and H. Su, "Diffusion dynamics models with generative state estimation for cloth manipulation," *arXiv preprint arXiv:2503.11999*, 2025.

[34] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *NeuRIPS*, 2017.

[35] W. F. Whitney, J. Varley, D. Jain, K. Choromanski, S. Singh, and V. Sindhwani, "Modeling the real world with high-density visual particle dynamics," *arXiv preprint arXiv:2406.19800*, 2024.

[36] W. Huang, Y.-W. Chao, A. Mousavian, M.-Y. Liu, D. Fox, K. Mo, and L. Fei-Fei, "Pointworld: Scaling 3d world models for in-the-wild robotic manipulation," *arXiv preprint arXiv:2601.03782*, 2026.

[37] A. Khazatsky, K. Pertsch, S. Nair, A. Balakrishna, S. Dasari, S. Karamcheti, and et al., "Droid: A large-scale in-the-wild robot manipulation dataset," *arXiv preprint arXiv:2403.12945*, 2024.

[38] N. Hansen, X. Wang, and H. Su, "Temporal difference learning for model predictive control," in *ICML*, 2022.

[39] G. Zhou, H. Pan, Y. LeCun, and L. Pinto, "Dino-wm: World models on pre-trained visual features enable zero-shot planning," *arXiv preprint arXiv:2411.04983*, 2024.

[40] M. Assran, A. Bardes, D. Fan, Q. Garrido, R. Howes, M. Muckley, A. Rizvi, C. Roberts, K. Sinha, A. Zholus *et al.*, "V-jepa 2: Self-supervised video models enable understanding, prediction and planning," *arXiv preprint arXiv:2506.09985*, 2025.

[41] W. Yuan, T. Khot, D. Held, C. Mertz, and M. Hebert, "Pcn: Point completion network," in *3DV*, 2018.

[42] Z. Huang, Y. Yu, J. Xu, F. Ni, and X. Le, "Pf-net: Point fractal network for 3d point cloud completion," in *CVPR*, June 2020.

[43] W. Khademi and L. Fuxin, "Diverse shape completion via style modulated generative adversarial networks," in *NeuRIPS*, 2023.

[44] Y. Nie, J. Hou, X. Han, and M. Niessner, "Rfd-net: Point scene understanding by semantic instance reconstruction," in *CVPR*, 2021.

[45] J. Tang, X. Chen, J. Wang, and G. Zeng, "Point scene understanding via disentangled instance mesh reconstruction," *arXiv preprint arXiv:2203.16832*, 2022.

[46] H. Li, J. Dong, B. Wen, M. Gao, T. Huang, Y.-H. Liu, and D. Cremers, "Ddit: Semantic scene completion via deformable deep implicit templates," in *ICCV*, 2023, pp. 21 894–21 904.

[47] N. Carion, L. Gustafson, Y.-T. Hu, S. Debnath, R. Hu, D. Suris, C. Ryali, K. V. Alwala, H. Khedr, A. Huang, J. Lei, T. Ma, B. Guo, A. Kalla, M. Marks, J. Greer, M. Wang, P. Sun, R. Rädle, T. Afouras, E. Mavroudi, K. Xu, T.-H. Wu, Y. Zhou, L. Momeni, R. Hazra, S. Ding, S. Vaze, F. Porcher, F. Li, S. Li, A. Kamath, H. K. Cheng, P. Dollár, N. Ravi, K. Saenko, P. Zhang, and C. Feichtenhofer, "Sam 3: Segment anything with concepts," *arXiv preprint arXiv:2511.16719*, 2025.

[48] G. R. Team, A. Abdolmaleki, S. Abeyruwan, J. Ainslie, J.-B. Alayrac, M. G. Arenas, A. Balakrishna, N. Batchelor, A. Bewley, J. Bingham *et al.*, "Gemini robotics 1.5: Pushing the frontier of generalist robots with advanced embodied reasoning, thinking, and motion transfer," *arXiv preprint arXiv:2510.03342*, 2025.

[49] B. Çalli, A. Singh, A. Walsman, S. S. Srinivasa, P. Abbeel, and A. M. Dollar, "The ycb object and model set: Towards common benchmarks for manipulation research," *2015 International Conference on Advanced Robotics (ICAR)*, 2015.

[50] B. Zhang, L. Ke, A. W. Harley, and K. Fragkiadaki, "Tapip3d: Tracking any point in persistent 3d geometry," *NeurIPS*, 2025.

[51] X. Wu, L. Jiang, P.-S. Wang, Z. Liu, X. Liu, Y. Qiao, W. Ouyang, T. He, and H. Zhao, "Point transformer v3: Simpler faster stronger," in *CVPR*, 2024, pp. 4840–4851.

[52] T. Darcet, M. Oquab, J. Mairal, and P. Bojanowski, "Vision transformers need registers," *arXiv preprint arXiv:2309.16588*, 2023.

[53] P. J. Huber, "Robust estimation of a location parameter," in *Breakthroughs in statistics: Methodology and distribution*. Springer, 1992, pp. 492–518.

[54] G. Williams, A. Aldrich, and E. A. Theodorou, "Model predictive path integral control: From theory to parallel computation," *Journal of Guidance, Control, and Dynamics*, 2017.

[55] S. Tao, F. Xiang, A. Shukla, Y. Qin, X. Hinrichsen, X. Yuan, C. Bao, X. Lin, Y. Liu, T. kai Chan, Y. Gao, X. Li, T. Mu, N. Xiao, A. Gurha, V. N. Rajesh, Y. W. Choi, Y.-R. Chen, Z. Huang, R. Calandra, R. Chen, S. Luo, and H. Su, "Maniskill3: Gpu parallelized robotics simulation and rendering for generalizable embodied ai," *RSS*, 2025.

[56] A. Mandlekar, S. Nasiriany, B. Wen, I. Akinola, Y. Narang, L. Fan, Y. Zhu, and D. Fox, "Mimicgen: A data generation system for scalable robot learning using human demonstrations," *arXiv preprint arXiv:2310.17596*, 2023.

[57] Z. Jiang, Y. Xie, K. Lin, Z. Xu, W. Wan, A. Mandlekar, L. J. Fan, and Y. Zhu, "Dexmimicgen: Automated data generation for bimanual dexterous manipulation via imitation learning," in *ICRA*. IEEE, 2025, pp. 16 923–16 930.

[58] Y. J. Ma, W. Liang, G. Wang, D.-A. Huang, O. Bastani, D. Jayaraman, Y. Zhu, L. Fan, and A. Anandkumar, "Eureka: Human-level reward design via coding large language models," *arXiv preprint arXiv: Arxiv-2310.12931*, 2023.

[59] Y. J. Ma, W. Liang, H. Wang, S. Wang, Y. Zhu, L. Fan, O. Bastani, and D. Jayaraman, "Dreureka: Language model guided sim-to-real transfer," in *Robotics: Science and Systems (RSS)*, 2024.

[60] G. Comanici, E. Bieber, M. Schaekermann, I. Pasupat, N. Sachdeva, I. Dhillon, M. Blistein, O. Ram, D. Zhang, E. Rosen *et al.*, "Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities," *arXiv preprint arXiv:2507.06261*, 2025.

[61] J. Gu, F. Xiang, X. Li, Z. Ling, X. Liu, T. Mu, Y. Tang, S. Tao, X. Wei, Y. Yao, X. Yuan, P. Xie, Z. Huang, R. Chen, and H. Su, "Maniskill2: A unified benchmark for generalizable manipulation skills," in *International Conference on Learning Representations*, 2023.

[62] X. Wu, D. DeTone, D. Frost, T. Shen, C. Xie, N. Yang, J. Engel, R. Newcombe, H. Zhao, and J. Straub, "Sonata: Self-supervised learning of reliable point representations," in *CVPR*, 2025, pp. 22 193–22 204.

[63] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, "FlashAttention: Fast and memory-efficient exact attention with IO-awareness," in *NeuRIPS*, 2022.

[64] T. Dao, "FlashAttention-2: Faster attention with better parallelism and work partitioning," in *ICLR*, 2024.

[65] Y. Zhu, A. Joshi, P. Stone, and Y. Zhu, "Viola: Imitation learning for vision-based manipulation with object proposal priors," in *CORL*, 2022.

[66] R. Wang, S. Xu, Y. Dong, Y. Deng, J. Xiang, Z. Lv, G. Sun, X. Tong, and J. Yang, "Moge-2: Accurate monocular geometry with metric scale and sharp details," vol. 38, 2025, pp. 35 928–35 959.

[67] H. Lin, S. Chen, J. Liew, D. Y. Chen, Z. Li, G. Shi, J. Feng, and B. Kang, "Depth anything 3: Recovering the visual space from any views," *arXiv preprint arXiv:2511.10647*, 2025.

VIII. Appendix

A. Data Generation

Point completion. We consider 100 random viewpoints for each object from the YCB dataset from which we obtain the partial point clouds inputs. A sample of the random views on three YCB objects is shown in Figure 6.



Fig. 6: *Point completion Data Generation*: Given the mesh/point cloud of the object, we render depth maps from 100 random views around a sphere of 1 unit looking at $(0, 0, 0)$, with the object in centered at the origin.

Dynamics modeling. For all tasks across ManiSkill and DexMimicGen, we write a custom wrapper to obtain the complete point clouds from the simulation itself by sampling points from objects/parts/geom meshes. Further, to handle point-to-point correspondences, we only sampled the points at the cannonical frame and applied rigid body pose transformations for all the desired entities in the scene. This significantly boosted the training data collection time as well as reduced the need to store point clouds at each step (only storing one cannonical point cloud and `len(traj)` transformation matrices).

B. Model Details & Hyperparameters

3DPWM dynamics modeling. For training the dynamics model, we adopt the PTV3 architecture [51], replacing batch normalization with layer normalization following Sonata [62]. To better suit the smaller dataset size, we reduce the depth of each of the four PTV3 encoding stages from the default configuration to `[2,2,2,2]`. Relative positional encoding (RPE) helps to capture spatial relationships between the points, but enabling it throughout the network makes it hard to use FlashAttention [63], [64] to speed up the training process. Instead, we enable RPE in alternating encoder and decoder blocks. Accordingly, we adjust the patch size for neighborhood computation from 256 in the layers where RPE is disabled to 48 when RPE is enabled. The bottleneck per-point feature dimension is set to $d = 256$, while the number of input points M depends on the grid resolution and scene complexity, typically ranging from 200 to 400 points. For action conditioning, both the `CrossAttn` and `SelfAttn` modules consist of three layers with eight attention heads each. The decoder outputs 128-dimensional per-point features, which are passed through a four-layer MLP to predict per-point velocities. Wherever applicable, we employ FlashAttention [63], [64] across all attention modules in PTV3, including the cross- and self-attention layers, to improve training efficiency. We provide detailed hyperparameters for the completion module, dynamics training as well as for planning in Appendix VIII-B. We outline the hyperparameters for all the module of 3DPWM in Table III.

Config/Hyperparameter	Value
Point Completion	
jitter prob	0.3
jitter mean	0.01
jitter variance	0.03
global feature channels	512
encoder channels	[32, 64, 128, 256]
decoder upsample factors	[2, 2, 2]
decoder hidden channels	64
seed generator blocks	4
seed generator - attn heads	8
seed generator - attn dims	384
seed generator - upsample factor	2
seed generator - output dim	256
Point Dynamics model	
serialization pattern	Z + TZ + H + TH
patch interaction	Shift Order + Shuffle Order
positional encoding	[CPE, RPE, CPE, RPE]
stride	[2, 2]
encoder depth	[2, 2, 2]
encoder channels	[32, 128, 256]
encoder num heads	[8, 8, 8]
encoder patch size	[256, 48, 256]
decoder depth	[2, 2]
decoder channels	[128, 256]
decoder num heads	[8, 8]
decoder patch size	[256, 48]
mlp ratio	4
qkv bias	True
drop path	0.3
pre norm	True
shuffle orders	True
MPPI Planner	
planning horizon	20-30 steps
# num samples	200
# top candidates	64
lambda_	0.7
noise σ	5
Training Hyperparameters	
loss	HuberLoss, delta=0.25
optimizer	AdamW
learning rate	0.0001
scheduler	StepLR, step_size=30, $\gamma = 0.1$

TABLE III: Hyperparameter values for different modules in our method.

SAM3 Prompting: Given the task description, we first prompt Gemini Robotics 1.5 VLM [48] to identify task-relevant objects in the scene by outputting their 2D point

locations. We then use these points as prompts to SAM3 [47] to segment the corresponding objects.

We use the following prompt to instruct the vision-language model to identify task-relevant objects and provide 2D point prompts for segmentation:

```
You are a vision-language model assisting a
    robot with a manipulation task.

Given:
1. A natural-language task description.
2. An RGB image of the current scene.

Your goal is to identify all objects that
   are relevant to successfully
completing the task and provide one or more
    2D point locations for each
relevant object.

Instructions:

1. Read the task description carefully and
   determine which physical
objects the robot must interact with,
   manipulate, move, avoid, or
otherwise reason about to complete the task
   .

2. In the image, locate each task-relevant
   object.

3. For each relevant object, select a 2D
   point that lies clearly INSIDE
the visible region of that object.
   - The point should be near the center of
     the visible portion of the
     object whenever possible.
   - Do not place the point on the object's
     boundary.
   - Do not place the point on the
    background, another object, or an
     occluding object.
   - Prefer points that are visually
    unambiguous and far from object
     boundaries.
   - If an object is partially occluded,
    place the point on the visible
     portion of the object.

...

Output format:

[
  {
    "object": "<short object description>",
    "point": [x, y]
  },
  ...
]

Example:

Task: "Pick up the red mug and place it
   inside the bowl."

Output:
[
  {
    "object": "red mug",
    "point": [312, 241]
  },
  {
    "object": "bowl",
    "point": [487, 326]
  }
]

Be precise: every point must fall inside
   the corresponding object's
visible pixels in the image.
```

C. Cost Function Specification

One of the key goals of learning a point based world model in the observation space, is to be able to specify cost functions to enable planning. This requires that the rollouts produced by the dynamics model are of high fidelity – so that the cost/reward computation on the rolled out point clouds is accurate. Since we perform point completion, which enables high fidelity long-horizon rollouts, we can use cost functions which can be defined on point clouds directly. We prompt Gemini 2.5-Flash [60] with the following prompt to obtain cost functions that can be defined in the point cloud observation space.

```
    You are a vision-language model
    assisting a robot with a manipulation
    task.

Your goal is to generate a task-specific
    cost function for evaluating predicted
    robot states during trajectory planning
    .

You are given:

1. A natural-language task description:

   <TASK>

2. A JSON file containing objects
    identified as potentially relevant to
    the task:
   <OBJECTS_JSON>

The objects JSON contains the objects
    detected in the scene and their 2D
    image point locations. It may contain:

* Objects that must be manipulated to
    complete the task.
* Objects that serve as destinations or
    targets.
```

````
* Objects that provide contextual
    information needed to complete the task
    .
* Objects that are not explicitly mentioned
     in the task but may be relevant
    because the robot could collide with
    them or they may constrain the robot's
    motion.

The generated cost function will be
    evaluated on 3D point clouds predicted
    by a robot world model. The point
    clouds represent the predicted geometry
     of objects and robot components at
    future states along candidate
    trajectories.

### Cost function interface

Generate a Python/PyTorch function with the
     following interface:

```python
def cost_fn(cfg, point, info):
    """
    Task-specific cost function.

    Args:
        cfg: Configuration object. In
    particular, cfg.device specifies
             the PyTorch device on which
    tensors should be created.

        point: Dictionary containing
    predicted 3D point clouds for
               objects and robot components
    . Keys correspond to object
               names from the provided
    objects JSON and may also include
               robot components such as '
    left_gripper' and
               'right_gripper'.

               Each point cloud is a
    PyTorch tensor containing 3D
               coordinates.

        info: Dictionary containing
    additional task/environment
              information that may be
    required to evaluate the task,
              such as goal positions, grasp
    /contact state, or task
              stage. Additional information
     is available under
              info['extra'] when provided.

    Returns:
        A single scalar PyTorch cost. Lower
     cost indicates a more
        desirable state.
    """
    ...
    return total_cost
```
````

````
### Requirements

1. **Infer the task objective**

   Carefully interpret `&lt;TASK&gt;` and
    determine what the robot must
    accomplish, including any intermediate
    stages or object relationships required
     for successful completion.

2. **Use the objects JSON**

   Use the provided objects to determine
    which objects should influence the cost
    .

   Do not invent task objects that are not
    present in the JSON.

3. **Include potentially relevant objects**

   Do NOT include clearly irrelevant scene
    elements such as the background or
    image artifacts. However, include
    objects that could potentially be
    relevant to execution, even if they are
     not explicitly mentioned in the task.

   In particular, consider objects that the
     robot arm, gripper, or manipulated
    object could collide with or that could
     constrain the robot's motion.

4. **Operate on predicted point clouds**

   The cost function should primarily
    operate on the 3D point clouds in `
    point`.

   For example, object geometry may be
    summarized using:

   * Point-cloud centroids.
   * Distances between point clouds.
   * Distances between object centers.
   * End-effector-to-object distances.
   * Object-to-target distances.
   * Relative positions.
   * Spatial overlap or containment.
   * Proximity to potential obstacles.

   Choose geometric quantities appropriate
    for the specific task.

5. **Use `info` when necessary**

   Use `info` for information that cannot
    be obtained reliably from the point
    clouds, such as:

   * Whether an object is currently grasped
    .
   * Task stage.
   * Goal positions.
````

```
   * Other task-specific environment state.

   Do not assume privileged information is
    available unless it is provided through
     `point` or `info`.

6. **Measure task progress**

   The cost should provide a meaningful
    measure of progress toward task
    completion.

   For multi-stage tasks, the cost may
    change depending on the current task
    stage. For example, it may first
    encourage reaching an object and then
    encourage moving the object toward its
    destination after grasping.

7. **Collision avoidance**

   Include collision or near-collision
    costs when potentially relevant objects
     could interfere with the robot's
    execution.

   The collision cost should increase as
    the robot or manipulated object
    approaches a potential obstacle.

   Collision avoidance should complement,
    rather than overwhelm, the primary task
     objective.

8. **Smoothness**

   Prefer smooth cost functions that
    provide useful ranking information
    between candidate states, including
    when none of the candidates completely
    succeeds.

   Prefer continuous functions such as
    Euclidean distances, `torch.tanh`,
    exponentials, or other smooth distance-
    based penalties over hard binary
    thresholds when appropriate.

9. **Scalar output**

   The function must return a single scalar
     PyTorch tensor:

   ```python
   return total_cost
   ```

   Lower cost must correspond to a better
    state.

10. **Task-specific design**

    Do not produce a generic cost function.
     Construct the cost specifically from
    the provided task description and
    objects JSON.

11. **Keep it simple**

    Use only the terms necessary to capture
     the essential task objective and
    relevant collision constraints. Avoid
    unnecessarily complicated heuristics.

12. **PyTorch compatibility**

    Use PyTorch operations for geometric
    calculations so that the function can
    operate directly on the tensors in `
    point`.

Output format

Return:

1. A brief description of the task
    objective and the relevant object roles
    .

2. A brief description of the cost terms
    and what each term measures.

3. The complete Python/PyTorch
    implementation:

```python
def cost_fn(cfg, point, info):
    ...
    return total_cost
```

4. A brief list of assumptions about the
    available keys in `point` and `info`.

The final implementation must be directly
    usable as a task-specific cost function
     for ranking predicted robot
    trajectories.
```

Alternatively, manually constructed cost functions can also be used. We show an example of the cost function adapted in a very straightforward manner from ManiSkill [55] in a pytorch-like code 1.

```python
def cost_fn_pick_cube(cfg, point, info):
    """
    Cost function for the PickCube-v1 task.
    """

    goal_xyz =
        torch.tensor(info['extra']['goal_pos'],
        device=cfg.device)

    left_gripper_xyz = point['left_gripper']
    right_gripper_xyz = point['right_gripper']
    cube_xyz = point['cube']

    cube_center_xyz = cube_xyz.mean(dim=0)
    tcp_xyz = (left_gripper_xyz.mean(dim=0) +
               right_gripper_xyz.mean(dim=0)) / 2

```

```python
    tcp_to_cube_dist = torch.norm(cube_center_xyz -
        tcp_xyz, dim=0)
    reaching_reward = 1.0 - torch.tanh(5.0 *
        tcp_to_cube_dist)

    cube_to_goal_dist = torch.norm(cube_center_xyz
        - goal_xyz, dim=0)
    place_reward = 1.0 - torch.tanh(5.0 *
        cube_to_goal_dist)

    contact_reward = 1.0 if info['grasped'] else 0.0

    if info['grasped']:
        total_cost = -place_reward
    else:
        total_cost = -(reaching_reward +
            contact_reward)

    return total_cost
```

Listing 1: Cost function for the PickCube task that operates on **3DPWM**'s rolled out point clouds.

Since the notion of contacts is not baked in explicitly, we perform grasping when the gripper is within 2.5 cm of the object (i.e distance between `tcp_xyz` and `cube_center_xyz` is ≤ 0.025) and assume a success (i.e `info['grasped'] = True`). Alternatively, we can also try computing the intersection between the left-gripper, object and the right-gripper to determine a contact, however, in practice we found it hard to tune the threshold for detecting the contact. We leave addressing this issue in point world models as a direction for future work.

D. Failure Scenarios

Point completion. First we discuss some of the failures of point cloud completion module on real world objects. We show a sample of completion success as well as failures in Figure 7. The failure scenarios primarily arise due to a mismatch between the training partial point cloud distribution and the real-world partial point cloud distribution. As mentioned in Sec. VI, our current data generation pipeline for point cloud completion does not include occlusions cause by other objects in the scene. This is because we generate partial point clouds of the objects by considering individual meshes of the objects and generating depth maps from 100 random viewpoints and backprojecting the depth to obtain the point cloud. This process, by design, does not include various kinds of occlusions caused due to the presence of other objects. Further, the noisy generation of points outside the boundary as seen in Fig 7 is also a common failure point in the pipeline.

E. Additional Experiments on 3DPWM robustness towards lighting conditions and camera viewpoints

We also experimented with 2 additional camera viewpoints as well as changing the lighting conditions to test the robustness of **3DPWM**. It has been shown that policies trained with calibrated point clouds are generally more robust to such lighting and camera viewpoint changes [20], [65]. This was further corroborated by our results on `PickApple` shown in Figure 8. Specifically, we observe that **3DPWM** maintains its higher performance when compared with ParticleFormer^{+}.



Fig. 7: *Point completion success & failures*: We show few samples of the fine-tuned point cloud completion model. **(Top)** Successful point cloud completions on partial point clouds of apple and mustard bottle. (Bottom) Failure scenarios of the completion on Drill and Apple from the YCB dataset.

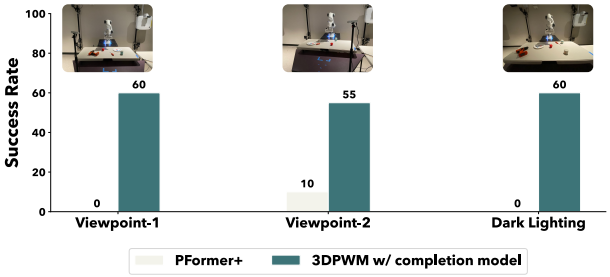


Fig. 8: *Lighting & Viewpoint condition*: We tested the robustness of **3DPWM** on `PickApple` task with two additional viewpoints as well as darker lighting condition. (Each plot is averaged over 20 trials.)

F. Additional Experiments: Comparison to GNN based methods

TABLE IV: Comparison between 3DPWM and graph neural network GBND

Task	Chamfer L1 (↓)		Reward Corr (r) (↑)	
	GBND [24]	3DPWM	GBND [24]	3DPWM
PickCube	0.185	0.009	0.37	0.86
Pick YCB	0.21	0.028	0.33	0.85
Box Cleanup	0.301	0.022	0.16	0.83

Gaussian Splatting based methods like [24] comprise of (a) Gaussian point-based dynamics and (b) splatting-based rendering. Since **3DPWM** focuses on modeling the dynamics and *not* the rendering, we compare against the GNN-based dynamics model from [24] in Table IV. As shown in [19] and confirmed by our results – GNN-based dynamics underperforms transformer-based approaches. Integrating a rendering pipeline into **3DPWM** remains a future work that we wish to address with **3DPWM** as the dynamics model.

TABLE V: Chamfer distance and reward correlation on both variants of ParticleFormer (+ and ∗) (Best and second best).

Task	Avg. rollout len.	Chamfer L1 (↓)			Reward Corr (r) (↑)		
		PFormer$^+$	PFormer*	3DPWM	PFormer$^+$	PFormer*	3DPWM
PickCube	66	0.080	0.132	0.009	0.61	0.41	0.86
Box Cleanup	218	0.132	0.225	0.022	0.30	0.19	0.83

G. Additional Experiments: Sim2Real in Cluttered Environment

We further evaluate the planning capabilities of **3DPWM** in cluttered environments across two tasks: `PickPlaceApple` and `StackCube`. In `PickPlaceApple`, the apple is initialized near a group of cluttered objects, and the cost function incorporates a heuristic penalty for collisions with surrounding objects. `StackCube` requires stacking a yellow cube on top of a blue cube in a moderately cluttered environment. This task is particularly challenging, as precise placement is critical; notably, all observed failures on `StackCube` are near-misses, as illustrated in Figure 9 (right).

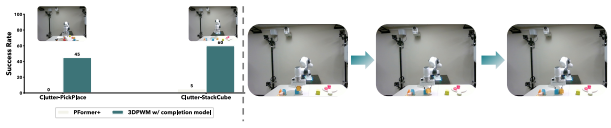


Fig. 9: *Sim2Real on Cluttered Environments*: **(Left)** We evaluate **3DPWM** on two cluttered-environment tasks: `PickPlaceApple`, in which the robot must pick an apple surrounded by clutter, and `StackCube`, in which the robot must stack cubes amid a cluttered scene. (Results are averaged over 20 trials per task.) **(Right)** Failure scenarios of **3DPWM** on `StackCube` occurs at the very end during placement of the yellow cube on top of the blue cube.

H. Baselines: Particleformer [19]

Here we lay out the specific details of our re-implementation of ParticleFormer [19]. Each partial point input has point velocities and one-hot vector representing instance segmentation. We run two architecture variants (i) ParticleFormer*, which is re-implementation of the the ParticleFormer baseline with 3 transformer layers as in the original paper, and (ii) ParticleFormer$^+$an equivalent variant of encoder and decoder that uses our PTV3 architecture [51]. Unlike **3DPWM**, we train both these variants on segmented partial point clouds and supervise the dynamics model with the hybrid loss (a combination of Chamfer and Hausdorff losses) as proposed by the authors.

We report the performance of ParticleFormer* here in Table V along with the copied results of ParticleFormer$^+$from Table I. We find that the original ParticleFormer with 3-layers of transformer is insufficient to capture the dynamics of both `PickCube` as well as `Box Cleanup` tasks and fares lower to ParticleFormer$^+$.

I. Environments

We evaluate our dynamics model design across 6 different tasks – `PickCube`, `StackCube` and `PickYCB` from the Maniskill simulator [55], `MugCleanup` from MimicGen [56] and `TwoArmBoxCleanup` and `TwoArmDrawerCleanup` from Dexmimicgen [57]. A sample snapshot of the environment is shown in Figure 10.

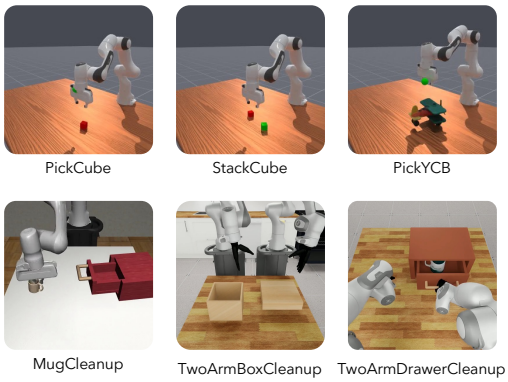


Fig. 10: *Environments used to evaluate* **3DPWM**:

PickCube: Lift the cube from the table to the goal position in the air specified by a 3D point in space.
StackCube: Stack the red cube on top of green cube.
PickYCB: Pick up the YCB object from the table to the goal position in the air specified by a 3D point in space.
MugCleanup: Pick and place the mug in side the drawer and close it.
TwoArmBoxCleanup: Pick up the lid of the box using two dexterous hands and place it on the box.
TwoArmDrawerCleanup: Pick and place the mug using two dexterous hands in side the drawer and close it.

For all the tasks with parallel jaw gripper, we use the end-effector delta actions, and for dexterous tasks we use 24-dim end effector actions (6-dim for pose, and 6 for finger control on each arm).

Task	Observation Points	Action dim
`PickCube`	1200	7
`StackCube`	1500	7
`PickYCB`	1200	7
`MugCleanUp`	8192	7
`TwoArmBoxCleanup`	10000	24
`TwoArmDrawerCleanup`	12000	24

TABLE VI: Number of points for complete point cloud and action spaces of each of the tasks.

J. Additional experiments we tried that did not work.

In this section we lay out some of the additional experiments we tried but did not see success and believe that this would be helpful to share with the community to improve upon.

Using SAM3D instead of custom trained point completion. We experimented with SAM3D [26] but found that

the completions, while generalizable to various commonly found objects *do not align* with the partial input despite providing sensor depth from the camera (instead of using DepthAnything or other foundation depth models). This is a known issue (see Issues #79 and #162 on the SAM3D github repository.)

Using Depth Foundation models to avoid noisy completions. We also tested using MOGE-2 [66] and DepthAnything3 [67] since they have shown impressive performance for depth estimation. However, we observed that while the depth was very clean and so were the resulting backprojected point clouds – these models still suffered with two major challenges which made decision making difficult: (a) similar to SAM3D, these depth foundation models suffered with object misalignment in the real world, and (b) they tended to miss out smaller details (depending on the viewpoint of the object) of the objects that could potentially be crucial for manipulation.